3.5 Code reviews

Code reviews are a systematic quality assurance process where developers examine source code to identify errors, ensure quality, and facilitate knowledge sharing before the code is integrated into a shared codebase.

They are a cornerstone of modern software development, essential for creating robust and maintainable software.

Purpose and Benefits

The primary goals of code reviews extend beyond simple bug finding to broader team and project improvement:

- **Error Detection:** Catching bugs, logic problems, and security vulnerabilities early in the development lifecycle, when they are less costly to fix.
- **Improved Code Quality & Consistency:** Ensuring the code adheres to established coding standards, style guidelines, and design principles, which makes the entire codebase more readable and maintainable.
- **Knowledge Transfer:** Sharing insights, design patterns, and domain-specific knowledge across the team, which helps junior developers learn from senior ones and reduces information silos.
- **Increased Collaboration & Ownership:** Fostering a sense of shared responsibility and collaboration, leading to better team cohesion.
- **Performance Optimization:** Identifying performance bottlenecks that might be missed by automated tests.

The Code Review Process

While the process can vary by team, a typical, modern, change-based code review (often part of a pull request workflow in systems like GitHub or GitLab) generally follows these steps:

1. **Code Submission:** The author completes a unit of work and submits their changes, usually via a pull request (PR).
2. **Review Assignment:** One or more peer developers are assigned or automatically notified to review the code.
3. **Code Examination:** Reviewers analyze the code for functionality, design, complexity, test coverage, naming, and adherence to standards.

4. **Feedback and Discussion:** Reviewers provide comments and suggestions. The author and reviewer(s) discuss the feedback to reach an agreement on the necessary changes.
5. **Revisions:** The author revises the code based on the feedback.
6. **Approval and Merge:** Once all issues are addressed and the reviewers are satisfied, the code is approved (often with "LGTM" - "looks good to me") and merged into the main codebase.

Key Best Practices

Effective code reviews rely on a positive culture and specific practices:

- **Keep Pull Requests Small:** Reviewing fewer than 400 lines of code at a time generally leads to a higher defect discovery rate and faster review cycles.
- **Provide Constructive Feedback:** Focus comments on the code, not the coder. Be explicit about requested actions and provide reasons for suggestions.
- **Automate What You Can:** Use static analysis and Continuous Integration (CI) tools to check for style violations, syntax errors, and common vulnerabilities, allowing human reviewers to focus on architectural and logical issues.
- **Optimize for Team Throughput:** Aim for speedy reviews to prevent developers from having to context-switch repeatedly.
- **Write Clear PR Descriptions:** Provide context, a demo (e.g., screenshot or GIF), and setup instructions to help reviewers understand the changes quickly.

Tools

Many tools facilitate the code review process, often integrating with version control systems:

- **GitHub, GitLab, and Bitbucket:** These platforms offer robust, built-in pull request/merge request workflows that are central to modern, tool-assisted code reviews.
- **Automated Tools:** Tools like SonarQube, ESLint, and AI-powered agents automate routine checks for code quality, security vulnerabilities, and style adherence.